Types, Variables and Input

L11 - Implicit type conversions

Department of Computer Science
University of Pretoria

14 March 2024

- In C++, **cout** is part of the Standard Library's **iostream** header.
- It is used to perform output operations in C++, typically to the standard **output stream** (usually the console or terminal window).
- The term "**cout**" stands for "**character output**" and is pronounced "see-out."

Cout implementation in C++ code

```
1  #include <iostream>
2  int main() {
3      // Outputting a string
4      std::cout << "Hello, World!" << std::endl;
5      // Outputting a number
6      int num = 8;
7      std::cout << "The value of num is: " << num << std::endl;
8      return 0;
9  }
```

- **Comments** serve as documentation of C++ code, explaining its purpose, functionality, and any important details that might not be immediately obvious from the code itself.
- Well-written comments improve the *readability of your code*, making it easier for other developers (or even yourself, in the future) to understand what the code does and how it works.

In C++ programming, we have two types of comments. These are:

- **Single Line Comments:** denoted by // . These are used when you want to comment on a single line. The // tells the compiler to ignore everything after the symbol to the end of the line.

In C++ programming, we have two types of comments. These are:

- **Multi - Line Comments:** denoted by /* and */ pair. This type is used when you would like to comment on more than one line. Everything between the two symbols is ignored by the compiler.

Layout Rules

- Good and consistent code layout enhances **code readability**.
- If code is readable, it is inherently more understandable and consequently reliable and maintainable.

Pointers to create good layouts

- Use indentation and blank lines to reveal the subordinate nature of blocks of code.
- Each line which is part of the body of a control structure (if, while, dowhile, for, switch) is indented one tab stop from the margin of its controlling line.
- The same rule applies to function, struct, or union definitions, and aggregate initialisers.

Syntax Errors

- Syntax errors occur when the code does not conform to the syntax rules of the programming language.
- In C++, syntax errors prevent the compiler from understanding your code and result in compilation failures.
- Syntax errors are usually straightforward to identify because they violate the language's grammar rules.

Examples of Syntax Errors

- Missing semicolons
- Mismatched parentheses, braces, or brackets
- Misspelled keywords or identifiers

Code Example

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int cout = 5;
7      int team = 4;
8      cout << \n "The sum is "<< cout + team ;
9      return 0;
10 }
```

Study the above code. Identify any syntax errors (if any), and give reasons.

Semantic Errors

- Semantic errors occur when the code executes without crashing, but it does not produce the expected result due to incorrect logic or flawed understanding of the problem.
- Unlike Syntax errors, Semantic errors do not violate the syntax rules, so they do not result in compilation failures.

Examples of Semantic Errors

- Incorrect Calculations
- Type Mismatches
- Division by zero

Code Example

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int num1 = 5;
6      char num2 = '7';
7
8      int result = num1 + num2;
9
10     cout << "The sum is: " << result;
11     return 0;
12 }
```

Study the above code. Identify any semantic errors (if any), and give reasons.

Data types used in C++

- Type conversions, refer to the process of converting one data type into another in C++
- Type conversions can occur automatically (known as **implicit conversions**) or can be performed manually by the programmer (known as **explicit conversions** or typecasts)

Implicit Type Conversions

- An **Implicit type** conversion occurs automatically when the compiler detects that a value of one type needs to be converted into another type in a compatible context.
- This often happens when assigning a value of one data type to a variable of another data type, or when passing arguments to functions.

Implicit Type Conversion Code Example

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int num1 = 5;
6 //Implicit conversion from int to double
7     double num2 = num1;
8     return 0;
9 }
```

Explicit Type Conversions

- **Explicit type** conversion involves manually specifying the conversion from one data type to another using **casting operators**.
- This allows the programmer to override the default type conversion behavior and enforce the desired conversion explicitly.

Explicit Type Conversion Code Example

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5    double num1 = 5.5;
6    //Explicit conversion from double to int
7    int num2 = static_cast<int>(num1);
8
9      return 0;
10 }
```

- Here, the static cast operator is used to explicitly convert the double value 5.5 to an integer and assign it to the variable num2.

Narrowing or Widening Conversions

- **Type conversions can involve narrowing or widening conversions.**
- A **narrowing conversion** occurs when converting a data type with a larger range to a data type with a smaller range, potentially resulting in loss of data.

Narrow Type Conversion Code Example

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5    double num1 = 5.8;
6    // Narrowing conversion (implicit)
7    int num2 = num1;
8    return 0;
9  }
```

- In this example, converting a double to an int may result in truncation of the decimal part.

Widening Conversions

- A **widening conversion** occurs when converting a data type with a smaller range to a data type with a larger range, typically without loss of data.

Widening Type Conversion Code Example

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5    int num1 = 15;
6    // Widening conversion (explicit)
7     double num2 = static_cast<double>(num1);
8    return 0;
9  }
```

Things to note

- Narrowing conversions may lead to unintended behaviour or loss of precision.
- Using explicit type conversions through casting operators can help make the conversion process more transparent and explicit in the code, reducing the likelihood of errors.